COS314 - Assignment 1

Three Pathfinding Algorithm Comparison

Wall-E simulated with: DFS BFS A* in a 2D Grid

Name: Aidan Dawson - 24593542

March 21, 2026

1 Introduction

This report evaluates and compares three search algorithms, such as: Depth-First Search - **DFS**, Breadth-First Search - **BFS**, and A* Search. These algorithms are to search for the shortest path accordingly over an 100×100 grid, navigating as Wall-E from start node, $(20, 50)$, to the goal node, $(80, 50)$, There is a U-trap in the grid as well as random noise to ensure algorithms are tested over many cases seeded with an input seed value. They will be analyzed by their execution time, total nodes visited, as well as the length of the shortest path found.

2 Experimental Setup

2.1 Environment Configuration

The grid: 100×100:

- **Start Node:** $(20, 50)$
- **Goal Node:** $(80, 50)$
- **U-Trap:** The back wall is at $x = 50$ from $y = 30$ to 70, with walls spanning to $x = 30$ at $y = 30$ and $y = 70$
- **Random Noise:** 15% of **empty** cells are filled with obstacles based of seed value

2.2 Algorithm Implementation

- **Note:** The grid is just a simple integer grid, from which the nodes are created as the grid is traversed.
- **DFS:** The DFS algorithm uses a stack instead of a queue, with each node visited the traversable children are pushed to the stack, and the algorithm accordingly pops the nodes to traverse from the stack which leads to a DFS execution for pathfinding.
- **BFS:** With the BFS algorithm a queue structure is used to ensure the FIFO quality of the BFS algorithm is met when traversing, visiting all know children first before grandchildren.
- **A*:** A Priority queue, ensuring that the nodes with the best score are at the head of the cue making use of the distance from the root goal, the shortest path, and with Manhattan distance heuristic: $h(n) = |x_1 - x_2| + |y_1 - y_2|$

2.3 Metrics Collected

For each algorithm, the following were recorded:

- Path Cost (number nodes from Start to Goal on shortest path found)
- Nodes Visited (the total of nodes visited across the whole grid)
- Execution Time (the time it took the specific algorithm to find the optimal path - milliseconds - ms)
- Optimality (Is it the shortest path?)

3 Results

Table 1: Comparison of Search Algorithms on grid with seed value = 0

Metric	**DFS**	**BFS**	**A***
Path Cost (Distance)	775	105	105
Nodes Visited	3534	7396	1712
Execution Time (ms)	35.0000	23.0000	29.0000
Optimality Guaranteed?	No	Yes	Yes

4 Critical Analysis

4.1 The Search Space

The search space consists of a grid of $100 \times 100 = 10,000$ possible states, after which $8,500$ nodes are actually traversable as 15% noise has been added on the grid. Each node has an branching factor of b=4, up, right, down, left. The U-shaped obstacle ensured that there was backtracking instead of just going to goal node straight away. This was especially noticeable with the DFS algorithm first exploring total inside of the U-shaped obstacle and then breaking free from it.

4.2 Optimality

- **DFS:** This algorithm makes no guarantee for an optimal path. It searches deep, making use of a LIFO stack structure, and backtracks when a dead-end is reached or returns when the goal is found. The DFS path is typically a long winding path, as seen in the results, DFS produced a path of length 775 compared to the optimal paths 105.
- **BFS:** This algorithm guarantees the optimal path by exploring nodes level for level. Since the graph is unweighted, BFS ensures the shortest path.
- **A*:** Making use of the Manhattan distance as an heuristic, A* guarantees an optimal path as the heuristic $h(n)$ is admissible and consistent. Manhattan distance satisfies both conditions since diagonal traversal is not allowed, meaning it will never overestimate the true cost only making use of left/right/up/down as movements.

4.3 Efficiency

- **DFS:** The Depth wise exploration leads to very unpredictable behavior in terms of number of nodes visited before the path is found. It can either find the shortest path on the first depth wise search leading to very little nodes explored, or it can backtrack an unpredictable amount of times leading to a lot of nodes explored before correct result is found. In the results DFS expanded 3,534 nodes before finding an optimal path of 775 nodes.
- **BFS:** Because it explores surrounding children nodes in an expanding number, the BFS algorithm will almost always explore more nodes than necessary to find the

optimal path. Because it exhaustively explores the search space it has very bad efficiency although it does find the optimal path eventually with a high execution time of 23ms and nodes visited= $7,396$.

- **A*:** Being the most efficient due to the use of an **admissable** and consistent heuristic $h(n)$ the search is guided in the direction of the goal leaving unlikely paths unexplored and being able to find the shortest path with the fewest nodes expanded, $1,712$, through the search compared to DFS and BFS.

4.4 Path Suboptimality

- **DFS:** Due to DFS using a LIFO stack it only explores one direction before considering another direction. It also has no awareness of the search space or cost of traversal together with the U-shaped obstacle the DFS shortest path found in our results is 792% longer than the optimal paths of BFS and A*.

- **BFS:** Shows no Suboptimality on the unweighted graph. By exploring level by level as soon as it reaches the level that leads to the goal it will have found the optimal path. Ratio: 100% of Suboptimality.

- **A*:** Because the heuristic is admissable and consistent the A* algorithm displays no Suboptimality.

4.5 Nodes Visited Comparison

- **DFS:** DFS is totally unpredictable in the number of nodes expanded. Depending on the noise and obstacle, it can expand very little or a large portion of the grid. Due to the U-shaped trap and its lack of cost awareness, DFS expanded 3,534 nodes in our results, more than A* but fewer than BFS in this case - not always less than BFS.

- **BFS:** BFS has the highest nodes visited of the three algorithms due to its exhaustive searching strategy. It explores all neighbouring nodes at each level before going deeper, expanding in all directions no matter where the goal is located being its biggest drawback.

- **A*:** A* expands the least amount of nodes compared to the other two algorithms due to the admissible Manhattan distance heuristic guiding the search toward the goal. Nodes with a high $f(n) = g(n) + h(n)$ value are not considered, effectively pruning portions of the search space that are not likely to lead to the best path.

4.6 The h(n) Heuristic

Without the $h(n)$ heuristic algorithms like the DFS and BFS methods have no guidance in terms of which next step to take towards the goal, rather they explore nodes blindly leading to Suboptimality. The A* algorithm rather makes use of this consistent and admissible heuristic $f(n) = g(n) + h(n)$, where $g(n)$ is the shortest path from the Start node and $h(n)$ is the Manhatten distance estimate, making it an informed search aiding it in finding the optimal path whilst expanding the fewest nodes visited leading to a quicker execution and less resources used.

4.7 Seed Impact

The seed is used to determine where the noise is placed on the 2D grid. this affects the algorithms performance directly with DFS being the most sensitive to this change. For example a "lucky" distribution can let the DFS algorithm find the optimal path on the first depth wise execution but on the other hand can lead to the DFS only finding the optimal path after exploring thousands of nodes/paths. BFS is not as much affected as it explores its neighbors stepwise regardless the only difference being if the neighbor is now an obstacle it will simply just not be visited but its neighbors will most likely be visited. Leading to BFS always finding optimal path. A* is in certain terms resilient to the changes brought forward by the seed value because of its use of a heuristic to guide the search. Although there are cases where the noise clusters around the goal leading A* to explore more nodes due to heuristic not being able to account for this.

5 Conclusion

The report covered the differences of the three algorithms, DFS, BFS, and A*. Showed by the results DFS might be the easiest to execute but indeed does not guarantee an optimal path. DFS is also veery sensitive to the seed value, noise on grid, as it will alternate the amount of nodes visited to very high or low, and never having the optimal path. BFS can guarantee the optimal path but will always explore the most nodes in search of this optimal path, although this also means it is immune to the search space and noise it does mean that BFS is the most inefficient and resource heavy. A* had the best Efficiency by making use of a theoretical admissible and consistent heuristic the Manhattan heuristic. It could find the most optimal path with least amount of nodes visited making it the most efficient at finding the optimal path which is always does. Showcasing the benefits of using domain knowledge for your search strategy compared to uninformed searches like DFS and BFS each having its respective drawbacks.